

GAUTAM BUDDHA UNIVERSITY



SCHOOL OF ICT

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DETAILED SYLLABI (SEMESTER I-IV)

M.TECH. (COMPUTER SCIENCE AND ENGINEERING)

SPECIALIZATION IN

SOFTWARE ENGINEERING

ACADEMIC BATCH: 2026-2028 AND ONWARDS

SEMESTER I

Barber;

Advanced Database Management System			
Course Code:	CS521	Course Credits:	3
Course Category:	PCC	Course (U / P)	P
Course Year (U / P):	1P	Course Semester (U / P):	1P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Knowledge of database design			
2. A general understanding of database, design and dependency			
3. Understanding of different types of databases			
4. Knowledge of databases on the internet			
5. Application on enhanced database			
COURSE OUTCOMES			
At the end of the course the students should be able to:			
1. Basic knowledge and understanding of ER diagram and UML class diagram.			
2. Ability to apply functionality and Normalization in relational databases.			
3. Recognize and fetch data from object oriented, parallel and distributed databases.			
4. Use XML and understand unstructured data.			
5. Implement concept and deduction of enhanced database on different applications.			

UNIT I: INTRODUCTION TO DATABASE DESIGN

Entities, Attributes, Entity Sets, Relationships, Key Constraints, Participation Constraints, Weak Entities, UML Class Diagrams, Subclasses, Superclasses, Inheritance, Specialization, Generalization, Constraints and Characteristics of Specialization and Generalization Hierarchies, Modeling of UNION Types Using Categories, Representing Specialization and Generalization In UML Class Diagrams, Data Abstraction, Knowledge Representation and Ontology Concepts.

UNIT II: DATABASE DESIGN THEORY

Problems Caused by Redundancy, Decompositions, Problems Related to Decomposition, Reasoning About FD's, FIRST, SECOND, THIRD Normal Form, BCNF, Fourth Normal Form, Lossless Join Decomposition, Dependency Preserving Decomposition, Schema Refinement in DataBase Design, Multi-Valued Dependencies.

UNIT III: OBJECT- ORIENTED, PARALLEL AND DISTRIBUTED DATABASES

Overview of Object-Oriented Concepts, Object Identity, Object Structure, Type Constructor, Encapsulation of Operations, Methods and Persistence; Architectures for Parallel Databases, Parallel Query Evaluation, Parallelizing Individual Operations, Sorting Joins, Distributed Database Concepts, Data Fragmentation, Replication and Allocation Techniques for Distributed Database Design, Query Processing in Distributed Databases, Concurrency Control and Recovery in Distributed Databases.

UNIT IV: DATABASES ON THE WEB AND SEMI-STRUCTURED DATA

Web interface, XML, structure of XML data, querying XML data, storage of XML data, XML applications, semi- structured data model, indexes for text data.

36th BoS Date: 30-July-2026

Academic Batch: 2026 and Onwards

UNIT V: ENHANCED DATA MODELS FOR ADVANCED APPLICATIONS

Active database concepts, temporal database concepts, spatial databases: concept and architecture, deductive databases and query processing, mobile databases, Geographic Information Systems (GIS).

Textbooks/ References Books:

1. Elmasri and Navathe, Fundamentals of Database Systems
2. Ramakrishnan and Gehrke, Database Management Systems
3. Korth, Silberschatz, Sudarshan, Database System Concepts
4. Rob and Coronel, Database Systems: Design, Implementation and Management
5. Date and Longman, Introduction to Database Systems

Handwritten signatures and initials in blue ink.

Advanced Data Structure and Algorithm			
Course Code:	CS523	Course Credits:	4
Course Category:	PCC	Course (U / P)	P
Course Year (U / P):	1P	Course Semester (U / P):	1P
No. of Lectures + Tutorials (Hrs/Week):	03 + 01	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45 + 15	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Understanding the appropriate data structures, ADT libraries, and using them to design algorithms for a specific problem.			
2. Be capable of solving problems using abstraction techniques.			
3. Be able to choose appropriate algorithms for a specific problem.			
4. Be able to analyse algorithms in terms of their efficiency and correctness.			
5. To accept the recent developments in the area of algorithm design.			
COURSE OUTCOMES			
At the end of the course, the students should be able to:			
1. Design and analyse programming problem statements.			
2. Choose appropriate data structures and algorithms for a specific problem.			
3. Gain the necessary mathematical abstraction to solve problems.			
4. Come up with an analysis of efficiency and proof of correctness.			
5. Comprehend and select algorithm design approaches in a problem-specific manner.			

UNIT I: INTRODUCTION

Review of Basic Concepts: Abstract data types, Data structures, Algorithms, Big-Oh, Small-Oh, Omega, Small-Omega and Theta Notations, finding time complexity of programs, Recurrence Relations: Solving Recurrence Relations, Substitution Method, Master Theorem.

UNIT II: HASHING

Hashing: Review of Hashing, Hash Function, Collision Resolution Techniques in Hashing, Separate Chaining, Open Addressing, Linear Probing, Quadratic Probing, Double Hashing, Rehashing, Extendible Hashing, Recent Trends in Hashing.

UNIT III: TREES & GRAPH

Trees: Binary Search Trees, AVL Trees, Red Black Trees, 2-3 Trees, B-Trees, Splay Trees, Minimum Spanning Tree (MST), Kruskal's Algorithm and Prim's Algorithm, Applications to MST.

Graph: Graph, Breadth First Search, Depth First Search, Shortest path in edge-weighted case (Dijkstra's), Bellman Ford Algorithms, Topological Sorting.

UNIT IV: SELECTED TOPICS

Strassen's Matrix Multiplication, Greedy method VS Dynamic Programming, Job sequencing with deadlines, Fractional Knapsack Problem, 0/1 Knapsack Problem, Travelling Salesman Problem, Huffman coding, Pre order, Post order, Inorder traversal, Postfix to infix notation, Infix to Postfix notation.

UNIT V: RECENT TRENDS

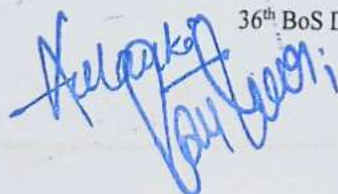
Linear Programming: Geometry of the feasibility region and Simplex algorithm.

NP-completeness: Examples, proof of NP-hardness and NP-completeness.

Recent Trends: Recent Trends in problem solving paradigms using recent searching and sorting techniques by applying recently proposed data structures.

36th BoS Date: 30-July-2026

Academic Batch: 2026 and Onwards





Text Books/ Reference Books:

1. Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
2. Algorithms Unlocked: Thomas H. Cormen.
3. The Algorithm Design Manual, Steven S. Skiena.
4. Algorithms: Robert Sedgewick and Kevin Wayne.
5. Advanced Data Structures: Peter Brass.

Handwritten signatures and initials:
A large signature "Arun" is written over the list.
To its right is another signature "Sankar".
Below "Arun" is a checkmark.
To the right of "Sankar" are initials "D".

Research Techniques in ICT			
Course Code:	CS525	Course Credits:	3
Course Category:	PCC	Course (U / P)	P
Course Year (U / P):	1P	Course Semester (U / P):	1P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. To understand the fundamentals, purpose, types, and processes of research in Information and Communication Technology.			
2. To develop the ability to identify, formulate, and define research problems and objectives.			
3. To understand and apply statistical, mathematical, and analytical techniques used in ICT research.			
4. To understand algorithmic research, experimental design, modelling, optimization, and validation techniques.			
5. To understand the role of simulation and computational techniques in conducting and evaluating ICT research.			
COURSE OUTCOMES			
At the end of the course, the students should be able to:			
1. Explain the fundamentals of research methodology, research processes, and research problem formulation.			
2. Apply appropriate statistical and analytical techniques to analyse research data.			
3. Develop and validate mathematical and computational models for ICT research problems.			
4. Design and evaluate algorithmic and experimental approaches for solving research problems.			
5. Apply simulation and computational techniques to conduct, analyse, and report ICT research studies.			

UNIT I: INTRODUCTION TO RESEARCH TECHNIQUES

Meaning of research, objectives of research, motivation in research, types of research, characteristics and prerequisites of research, significance of research, research process, sources of research problem, criteria of identifying the problem, necessity of defining the problem, errors in selecting research problem, technique involved in defining the problem, report, and paper writing.

UNIT II: DATA ANALYSIS AND STATISTICAL TECHNIQUES

Data and their analyses, quantitative methods and techniques, Measure of central tendency, measures of variation, frequency distribution, analysis of variance, methods, Correlation analysis, regression analysis, time series and forecasting, introduction to discriminant analysis, factor analysis, cluster analysis, conjoint analysis, probability distribution, binomial distribution, poisson distribution, uniform distribution, exponential distribution, and normal distribution, sampling methods, test of hypothesis.

UNIT III: MATHEMATICAL MODELING

Steps of modelling, operations research models like queuing theory, stochastic processes, application of models, conceptual framework development and validation techniques, optimization techniques.

UNIT IV: ALGORITHMIC RESEARCH

Algorithmic research problems, types of algorithmic research, types of solution procedure, steps of algorithm development, steps of algorithmic research, design of experiments.

Handwritten signatures and initials in blue ink.

36th BoS Date: 30 July-2026

Academic Batch: 2026 and Onwards

UNIT V: SIMULATION AND SOFT COMPUTING TECHNIQUES

Introduction to soft computing, artificial neural network, genetic algorithm, fuzzy logic and their applications, tools of soft computing, need for simulation, types of simulation, simulation language, fitting the problem to simulation study, simulation models, output analysis, data simulation packages like MATLAB, NS2, ANSYS, Cadence

Text books/ Reference Books:

1. Research Methodology: Methods and Techniques, C.R. Kothari
2. Research Methodologies, R. Panneerselvam, Prentice Hall, 2007.
3. Research in Education, Best John V. and James V Kahn, Wiley eastern, 2005.
4. Elements of Educational Research, Sukhia, S.P., P.V. Mehrotra, and R.N. Mehrotra, PHI publication, 2003.
5. Methodology of Research Education, K. Setia, IEEE publication, 2004.
6. Research methodology, Methods and Techniques, Kothari, C.R., 2000.

Handwritten signatures:
 Helanki
 Panneerselvam
 P
 W

Python Programming			
Course Code:	CS527	Course Credits:	3
Course Category:	PCC	Course (U / P)	P
Course Year (U / P):	1P	Course Semester (U / P):	1P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. To understand the fundamental concepts, syntax, and programming constructs of the Python programming language.			
2. To develop Python programs using conditional statements, loops, expressions, and basic input/output operations.			
3. To understand and apply functions, strings, and built-in data structures such as lists, tuples, sets, and dictionaries in Python programming.			
4. To develop Python programs using file handling, exception handling, modules, abstract data types, and object-oriented programming concepts.			
5. To apply recursion, searching, sorting, and merging techniques to solve computational problems using Python.			
COURSE OUTCOMES			
At the end of the course, the students should be able to:			
1. Develop and execute Python programs using basic syntax, expressions, operators, and input/output operations.			
2. Apply conditional statements and looping constructs to solve computational problems using Python.			
3. Develop modular Python programs using functions, strings, and built-in data structures such as lists, tuples, sets, and dictionaries.			
4. Implement file handling, exception handling, modules, and object-oriented programming concepts in Python.			
5. Apply recursion, searching, sorting, and merging techniques to solve computational problems efficiently.			

UNIT I: INTRODUCTION

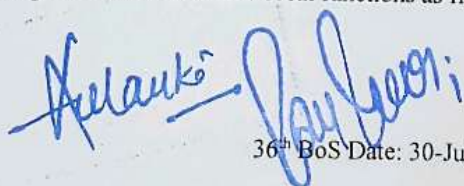
The Programming Cycle for Python, Python IDE, Interacting with Python Programs, Elements of Python, Type Conversion. Basics: Expressions, Assignment Statement, Arithmetic Operators, Operator Precedence, Boolean Expression.

UNIT II: CONDITIONALS

Conditional statement in Python (if-else statement, its working and execution), Nested-if statement and Elif statement in Python, Expression Evaluation & Float Representation. Loops: Purpose and working of loops; while loop, including its working; For Loop; Nested Loops; Break and continue.

UNIT III: FUNCTION

Parts of A Function, Execution of A Function, Keyword and Default Arguments, Scope Rules. Strings: Length of the string and perform Concatenation and repeat operations in it. Indexing and Slicing of Strings. Python Data Structures: Tuples, Unpacking Sequences, Lists, Mutable Sequences, List Comprehension, Sets, Dictionaries. Higher Order Functions: Treat functions as first-class Objects, Lambda Expressions


36th BoS Date: 30-July-2026

Academic Batch: 2026 and Onwards

UNIT IV: FILE I/O

File input and output operations in Python Programming. Exceptions and Assertions Modules: Introduction, Importing Modules, Abstract Data Types: Abstract data types and ADT interface in Python Programming. Classes: Class definition and other class operations, Special Methods (such as `_init_`, `_str_`, comparison methods, and Arithmetic methods), Class Example, Inheritance, Inheritance and OOP.

UNIT V: ITERATORS & RECURSION

Recursive Fibonacci, Tower of Hanoi, Search: Simple Search and Estimating Search Time, Binary Search and Estimating Binary Search Time, Sorting & Merging: Selection Sort, Merge List, Merge Sort, Higher Order Sort

Text books/ Reference Books:

1. Allen B. Downey, "Think Python: How to Think Like a Computer Scientist", 2nd edition, Updated for Python 3, Shroff/O'Reilly Publishers, 2016 (<http://greenteapress.com/wp/thinkpython/>)
2. Guido van Rossum and Fred L. Drake Jr, —An Introduction to Python – Revised and updated for Python 3.2, Network Theory Ltd., 2011.
3. John V Guttag, —Introduction to Computation and Programming Using Python –, Revised and expanded Edition, MIT Press, 2013

Handwritten signatures:
Hilanko, Jayaram, P, [unintelligible]

Energy and Environment			
Course Code:	ES415	Course Credits:	3
Course Category:	OEC	Course (U / P)	P
Course Year (U / P):	1P	Course Semester (U / P):	1P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. To provide in-depth knowledge of renewable and non-renewable energy resources			
2. To provide knowledge about energy harnessing techniques			
3. To study the energy-environment issues			
COURSE OUTCOMES			
At the end of the course, the students can use the knowledge so gathered and utilized to:			
1. Meet the challenges of energy vis-a-vis environmental security			

UNIT I: SUN AS SOURCE OF ENERGY

Nature of its radiation, solar radiation and its spectral characteristics; Conventional energy sources (coal, oil, biomass and natural gas), non-conventional energy sources (hydro-electric power, tidal, wind, geothermal, solar, nuclear magneto-hydrodynamic power MHD); Energy use pattern in India and parts of the world, Energy security.

UNIT II: FOSSIL FUELS

Classification, composition, physico-chemical characteristics; Calorific value – gross and net; Energy content of coal, petroleum and natural gas, shale oil, coal bed methane, gas hydrates. Concept of Green Energy; Principles of generation of hydropower, tidal energy, ocean thermal energy conversion, wind power, geothermal energy, solar energy (solar collectors, photo-voltaic modules, solar ponds).

UNIT III: NUCLEAR ENERGY

Fission and fusion, nuclear fuels, nuclear reactor – principles and types; Mechanism of radiation action on living organisms - Stochastic and Non-stochastic effects, delayed effects; Radioactivity from nuclear reactors, fuel processing and radioactive waste, hazards related to power plants.

UNIT IV: BIOENERGY

Types, importance, methods of energy production from biomass

UNIT V: ENVIRONMENTAL IMPLICATIONS OF ENERGY USE

CO₂ emission and atmosphere –scenario in developed and developing world (and India), Global warming, Radiative forcing, Impacts of large-scale exploitation of solar, wind, hydro, nuclear and bioenergy sources, National Solar Mission, National Mission for Enhanced Energy Efficiency, case studies.

TextBooks/ Reference Books:

1. Fay, J.A. and Golomb, D.S. 2011. Energy and the Environment, Oxford University Press, New Delhi.
2. Iqbal, M. 1983. An Introduction to Solar Radiation. Academic Press, New York.
3. Kaushika, N.D. and Kaushik, K. 2004. Energy, Ecology and Environment: A Technological Approach, Capital Publications, New Delhi. Website - <https://nptel.ac.in/course.html>

Handwritten signatures and initials:

36th BoS Date: 30-July-2026

Academic Batch: 2026 and Onwards

Advanced Database Management System Lab			
Course Code:	CD581	Course Credits:	2
Course Category:	PCC	Course (U / P)	U
Course Year (U / P):	2U	Course Semester (U / P):	3U
No. of Labs (Hrs/Week):	1 (4 HRS)	Mld Sem. Exam Hours:	"
Total No. of Labs:	10	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. To develop practical skills in database creation, manipulation, and querying using relational database management systems.			
2. To implement database design concepts including data types, constraints, normalization, and relational operations.			
3. To develop and execute SQL queries using joins, aggregation, functions, and subqueries for retrieving and analysing data.			
4. To understand and implement database transaction management and query-processing concepts through practical exercises.			
5. To apply database concepts to solve practical data-management problems using SQL and relational database systems.			
COURSE OUTCOMES			
At the end of the course, the students should be able to:			
1. Develop and execute SQL statements for creating, modifying, and managing relational databases.			
2. Apply constraints, normalization, joins, and relational operations to design and manage database structures.			
3. Construct SQL queries using aggregation, functions, subqueries, and other query-processing techniques.			
4. Implement and analyse transaction-management concepts in relational database systems.			
5. Apply database concepts to solve practical data-management and analysis problems.			

List of Experiments:

1. Introduction to MySQL, PostgreSQL, and Microsoft SQL software.
2. An exercise on data types in PostgreSQL & Data Definition Language Commands
3. Exercise on Data Manipulation Language and Transaction Control Commands using PostgreSQL.
4. Exercise on Types of Data Constraints using PostgreSQL.
5. Exercise on JOINS (Single-Table) Using Normalization
6. Exercise on JOINS (Multiple-Table) Using Normalization
7. Exercise on GROUP BY/ORDER BY Clause and Date Arithmetic using PostgreSQL.
8. Exercise on different Functions (Aggregate, Math and String)
9. Exercise on different types of subqueries
10. Procedures, Views and Triggers

Handwritten signatures:
 Helankar, [Signature], [Signature]

Python Programming Lab			
Course Code:	CD583	Course Credits:	2
Course Category:	PCC	Course (U / P)	P
Course Year (U / P):	1P	Course Semester (U / P):	1P
No. of Labs (Hrs/Week):	1 (4 HRS)	Mid Sem. Exam Hours:	-
Total No. of Labs:	10	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Interpret the use of procedural statements like assignments, conditional statements, loops and function calls.			
2. Infer the supported data structures like lists, dictionaries and tuples in Python			
3. Illustrate the application of matrices and regular expressions in building Python programs.			
4. Discover the use of external modules in creating Excel files and navigating the file systems.			
5. Describe the need for Object-oriented programming concepts in Python.			
COURSE OUTCOMES			
At the end of the course, the students should be able to:			
1. Write, test, and debug simple Python programs.			
2. Implement Python programs with conditionals and loops.			
3. Use functions for structuring Python programs.			
4. Represent compound data using Python lists, tuples, and dictionaries			
5. Read and write data from/to files in Python.			

List of Experiments:

1. Write a Python program to find the maximum of a list of numbers.
2. Write a Python program to perform Matrix Multiplication.
3. Write a Python program first n prime numbers
4. Write a Python program for selection sort.
5. Write a Python program to compute the GCD of two numbers.
6. Write a Python program to find the most frequent words in a text file.
7. Write a Python program to create a scientific calculator
8. Write a Python program to print all the Disarium numbers between 1 and 100.
9. Write a Python program to encrypt the text using the Caesar Cypher technique. Display the encrypted text.
Prompt the user for input and the shift pattern.
10. Write a Python program to construct a linked list. Prompt the user for input. Remove any duplicate numbers from the linked list.
11. Perform the following file operations using Python

Handwritten signatures and initials:

SEMESTER II

Sanjay

Advanced Software Engineering			
Course Code:	CS522	Course Credits:	3
Course Category:	PCC	Course (U / P)	P
Course Year (U / P):	1P	Course Semester (U / P):	2P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. To introduce software reuse — types, approaches, benefits/barriers — and the CBSE process.			
2. To build application systems from reusable components using application/component families, variability, façades and layered architecture.			
3. To impart knowledge of software change, evolution, maintenance and reengineering of legacy systems.			
4. To cover software dependability, safety, reliability and quality assurance (V&V) supported by CASE tools.			
5. To introduce usability engineering, HCI, and modern paradigms — Aspect-Oriented, Cleanroom, and AI/ML in the SDLC.			
COURSE OUTCOMES			
At the end of the course, the students should be able to:			
1. Implement reusability mechanisms for producing application systems.			
2. Explore maintenance and reengineering approaches for legacy systems.			
3. Understand the properties and methods of designing reliable and safe systems.			
4. Analyse and apply CASE in SDLC phases.			
5. Study of advanced concepts in various domains to produce an effective software system			

UNIT I: INTRODUCTION

Introduction to Software Reuse: What is Software Reuse?, Reuse types, Reuse Approaches, Reuse Technology, Reuse benefits & barriers, Reuse success & failure Factors, CBSE Process, Reuse Driven Software Engineering is a business.

UNIT II: ARCHITECTURAL CONCEPTS FOR SOFTWARE REUSE

Architectural Concepts in Reuse: Application and component systems, Application families allow significant reuse, Developing Application Systems from Reusable Components: Reuse Variability, Facades for Component System Internals and Externals. Organising a system in Layered Architecture.

UNIT III: CONCEPTS OF SOFTWARE CHANGE AND REENGINEERING

Software change, Software Evolution, Software maintenance: Models and Metrics, Reengineering: Reengineering Process and Activities: Program Comprehension, Reverse Engineering, Restructuring, Forward Engineering, Re-documentation. Software Ageing.

UNIT IV: SOFTWARE DEPENDABILITY, QUALITY, AND CASE TECHNOLOGY

Dependability attributes of reusable and component-based systems — availability and safety as design goals (concepts and trade-offs). Quality of reused components: quality factors and verification & validation at component and integration level. Computer-Aided Software Engineering (CASE): scope and technology, CASE support in the SDLC, second-generation CASE tools, architecture of a CASE environment.

UNIT V: USABILITY ENGINEERING

Usability Engineering: HCI, Types of UI, Component-Based GUI Development, Usability Engineering Process and Methods. Aspect-Oriented Software Engineering. Cleanroom Software Engineering and Crowdsourcing. Artificial Intelligence & Machine Learning in SDLC.

Text Books/Reference Books:

1. Software Reuse Architecture, Process and Organization for Business Success- by Ivar Jacobson, Martin Griss, Patrick Johnson, First Edition, Pearson Education, 2000.
2. Software Engineering: Concepts & Practices- Ugrasen Suman, Cengage Learning Publications, Second Ed. 2022.
3. Software Engineering- Ian Somerville, Pearson Education Asia, 10th Edition, 2016.
4. Software Engineering: A Practitioner's Approach- R. S. Pressman, Tata McGraw-Hill International Editions, New York.
5. Software Engineering: An Engineering Approach, by J.F. Peters and W. Pedrycz, Publisher: John Wiley and Sons.
6. Software Engineering: A Practitioner's Approach by Roger Pressman, Publisher: McGraw-Hill.
7. Fundamentals of Software Engineering by Ghezzi, Jayazeri, and Mandrioli, Publisher: Prentice-Hall.
8. Software Engineering Fundamentals by Ali Behforooz and Frederick J. Hudson, Publisher: Oxford University Press.

Apurva

R M

Varun;

Deep Learning			
Course Code:	CS524	Course Credits:	3
Course Category:	PCC	Course (U / P)	P
Course Year (U / P):	1P	Course Semester (U / P):	2P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Gain a historical perspective of Deep Learning and become familiar with the building blocks of neural networks — the perceptron, backpropagation, autoencoders and batch-normalisation.			
2. To understand the fundamental building blocks of neural networks, including artificial neurons, activation functions, loss functions, and network architectures.			
3. Apply gradient descent, backpropagation and regularisation techniques to train deep networks and control overfitting.			
4. Design and implement convolutional and recurrent architectures (CNNs, RNNs, LSTMs) for vision and sequence tasks.			
5. Explore advanced deep architectures such as GANs and transfer learning, and evaluate deep learning models across application domains.			
COURSE OUTCOMES			
At the end of the course, the students should be able to:			
1. Demonstrate knowledge of the building blocks of a Deep Learning system.			
2. Build and train neural networks using TensorFlow/Keras with appropriate activation and loss functions.			
3. Apply gradient descent, backpropagation and regularisation to optimise deep networks and mitigate overfitting.			
4. Design and apply convolutional and recurrent models (CNNs, RNNs, LSTMs) to image and sequence tasks.			
5. Evaluate and compare deep learning models — including GANs and transfer learning — across different application domains.			

UNIT I: INTRODUCTION TO DEEP LEARNING

History, capabilities, the perceptron, Neural network learning: Back-Propagation, Practical network training, Autoencoders, Batch-normalisation, Overfitting and generalisation.

UNIT II: INTRODUCTION TO TENSORFLOW AND ACTIVATION FUNCTIONS

Computational Graph, Key highlights, creating a Graph, Regression example, Gradient Descent, TensorBoard, Modularity, Sharing Variables, Keras Perceptrons: What is a Perceptron, XOR Gate, Sigmoid, ReLU, Hyperbolic Fns, Softmax. Artificial Neural Networks: Introduction, Perceptron Training Rule, Gradient Descent Rule.

UNIT III: DIMENSIONALITY REDUCTION AND CONVOLUTIONAL NETWORKS

Dimensionality reduction: Linear methods (PCA, LDA) and manifolds, metric learning, autoencoders for dimensionality reduction. Convolutional Networks: Introduction to Convnets, Architectures — AlexNet, VGG, Inception, ResNet; Training a Convnet: weight initialisation and hyperparameter optimisation.

UNIT IV: OPTIMIZATION AND GENERALIZATION IN DEEP LEARNING

Optimization in deep learning— Non-convex optimization for deep networks- Stochastic Optimization Generalization in neural networks- Spatial Transformer Networks- Recurrent networks, LSTM Recurrent Neural Network Language Models- Word-Level RNNs & Deep Reinforcement Learning - Computational & Artificial Neuroscience.

36th BoS Date: 30-July-2026

Academic Batch: 2026 and Onwards

UNIT V: APPLICATIONS OF DEEP LEARNING

Image-net- Detection-Audio WaveNet-Natural Language Processing Word2Vec - Joint Detection Bio-Informatics- Face Recognition- Scene Understanding- Gathering Image Captions.

Text Books/Reference Books:

1. Goodfellow, Yoshua Bengio, Aaron Courville, Deep Learning, MIT Press, 2016.
2. Deng & Yu, Deep Learning: Methods and Applications, Now Publishers, 2013.
3. Michael Nielsen, Neural Networks and Deep Learning, Determination Press, 2015.

Handwritten signature: Helanki Varun

DevOps Engineering			
Course Code:	CS526	Course Credits:	3
Course Category:	PCC	Course (U / P):	P
Course Year (U / P):	1P	Course Semester (U / P):	2P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Describe the agile relationship between development and IT operations.			
2. Understand the skill sets and high-functioning teams involved in DevOps and related methods to reach a continuous delivery capability.			
3. Implement automated system updates and the DevOps lifecycle.			
4. Discussion on DevOps Systems			
5. Evaluation of System design lifecycle in DevOps Engineering			
COURSE OUTCOMES			
At the end of the course, the students should be able to:			
1. Identify components of a DevOps environment.			
2. Describe Software development models and architectures of DevOps.			
3. Apply different project management, integration, testing and code deployment tools.			
4. Investigate different DevOps Software development models.			
5. Assess various DevOps practices.			
6. Collaborate and adopt DevOps in real-time projects.			

UNIT I: INTRODUCTION

Introduction, Agile development model, DevOps, and ITIL. DevOps process and Continuous Delivery, Release management, Scrum, Kanban, delivery pipeline, bottlenecks, examples.

UNIT II: SOFTWARE DEVELOPMENT MODELS AND DEV OPS

DevOps Lifecycle, DevOps, and Continuous Testing. DevOps influence on Architecture: Introducing software architecture, Architecture rules of thumb, The separation of concerns, Handling database migrations, Microservices, and the data tier, DevOps, architecture.

UNIT III: INTRODUCTION TO PROJECT MANAGEMENT

The history of source code management, Roles and code, source code management system and migrations, Shared authentication, Hosted Git servers, Different Git servers, Docker intermission, The pull request model.

UNIT IV: INTEGRATING THE SYSTEM

Build systems, managing build dependencies, Jenkins plug-ins, and file system layout, the host server, build slaves, Software on the host, Triggers, Job changing.

UNIT V: MONITORING & CLOUD IN DEVOPS

Monitoring Tools, Performance Monitoring, Cloud Computing, AWS Basics, DevOps in Cloud, Handling database, CI pipelines, Automated testing.

Textbooks/ Reference Books :

1. The DevOps Handbook – Gene Kim, Jez Humble, Patrick Debois, John Willis. Publisher: IT Revolution Press, Year: 2016 (first edition).
2. Continuous Delivery – Reliable Software Releases through Build, Test and Deployment Automation – Jez Humble, David Farley. Publisher: Addison-Wesley (or Addison Wesley Professional), Year: 2010.

36th Bos Date: 30-July-2026

Academic Batch: 2026 and Onwards

Blockchain Technology			
Course Code:	CS528	Course Credits:	4
Course Category:	PCC	Course (U / P)	P
Course Year (U / P):	1P	Course Semester (U / P):	2P
No. of Lectures + Tutorials (Hrs/Week):	03 + 01	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45 + 15	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. To get acquainted with the concept of Distributed ledger systems and Blockchain.			
2. To learn the concepts of consensus and mining in Blockchain through the Bitcoin network.			
3. To understand Ethereum and develop/deploy smart contracts using different tools and frameworks.			
4. To understand permissioned Blockchain and explore Hyperledger Fabric.			
5. To understand different types of crypto assets.			
COURSE OUTCOMES			
At the end of the course the students should be able to:			
1. Describe the basic and fundamental concept of Blockchain and Distributed Ledger Technology.			
2. Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions			
3. Implement smart contracts in Ethereum using different development frameworks.			
4. Develop simple applications using Solidity language on Ethereum platform.			
5. Interpret different Crypto assets and Crypto currencies			

UNIT I: FUNDAMENTALS OF BLOCKCHAIN TECHNOLOGY

Blockchain – Definition, architecture, elements of blockchain, benefits and limitations, types of blockchain.

Consensus – definition, types, consensus in blockchain.

Decentralization – Decentralization using blockchain, Methods of decentralization, Routes to decentralization, Blockchain and full ecosystem decentralization.

UNIT II: FUNDAMENTALS OF CRYPTOGRAPHY

Introduction to Cryptography, Symmetric cryptography – AES. Asymmetric cryptography – RSA. Elliptic curve cryptography, Digital signatures – RSA digital signature algorithms. Secure Hash Algorithms – SHA-256. Applications of cryptographic hash functions – Merkle trees, distributed hash tables.

UNIT III: CONSENSUS ALGORITHMS AND BITCOIN

Consensus Algorithms, Crash fault-tolerance (CFT) algorithms – Paxos, Raft. Byzantine fault-tolerance (BFT) algorithms – Practical Byzantine Fault Tolerance (PBFT), Proof of work (PoW), Proof of stake (PoS), Types of PoS. Bitcoin – Definition, Cryptographic keys – Private keys, public keys, addresses. Transactions – Lifecycle, coinbase transactions, transaction validation. Blockchain – The genesis block. Mining – Tasks of miners, mining algorithm, hash rate. Wallets – Types of wallets. .

UNIT IV: SMART CONTRACTS AND USE CASES

Smart Contracts – Definition, Smart contract templates, Oracles, Types of oracles, Deploying smart contracts. Decentralization terminology – Decentralized applications, Decentralized Autonomous Organizations. Use cases of Blockchain technology – Government, Health care, Finance, Supply chain management. Blockchain and allied technologies – Blockchain and Cloud Computing, Blockchain and Artificial Intelligence.

UNIT V: ETHEREUM AND SOLIDITY

Ethereum – The Ethereum network. Components of the Ethereum ecosystem – Keys and addresses, Accounts, Transactions and messages. The Ethereum Virtual Machine, Blocks and blockchain. The Solidity language – The layout of a Solidity source code, Structure of a smart contract, variables, data types, control structures, events, inheritance, libraries, functions, error handling. Smart contracts Case study: Voting, Auction.

Text Books/Reference Books:

1. Imran Bashir, Mastering Blockchain: A deep dive into distributed ledgers, consensus protocols, smart contracts, DApps, cryptocurrencies, Ethereum, and more, Packt Publishing, Third edition, 2020.
2. Ritesh Modi, Solidity Programming Essentials: A beginner's guide to build smart contracts for Ethereum and blockchain, Packt Publishing, First edition, 2018.
3. Kumar Saurabh, Ashutosh Saxena, Blockchain Technology: Concepts and Applications, First Edition, Wiley Publications, First edition, 2020.
4. Chandramouli Subramanian, Asha A George, et al, Blockchain Technology, Universities Press (India) Pvt. Ltd, First edition, August 2020.
5. Blockchain Technology: Concepts and Applications, Kumar Saurabh and Ashutosh Saxena, Wiley.

Handwritten signatures and initials:
Aelank
V
Saurabh; P

Advanced Software Engineering Lab			
Course Code:	CS582	Course Credits:	2
Course Category:	PCC	Course (U / P)	P
Course Year (U / P):	1P	Course Semester (U / P):	2P
No. of Labs (Hrs/Week):	1 (4 HRS)	Mid Sem. Exam Hours:	"
Total No. of Labs:	10	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Students will have a successful career in technology or managerial functions.			
2. Students will have a solid technical and professional foundation to continue higher studies.			
3. Students will have skills to develop products, offer services and create new knowledge.			
4. Students will have fundamental awareness of Industry processes, tools and technologies.			
COURSE OUTCOMES			
At the end of the course, the students should be able to:			
1. Identify, formulate, review research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences			
2. Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.			
3. Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.			
4. Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.			

List of Experiments:

1. Study the GitHub control.
2. Write down the problem statement for a suggested system of relevance
3. Do requirement analysis and develop Software Requirement Specification Sheet (SRS) for the suggested system.
4. To perform the function-oriented diagram: Data Flow Diagram (DFD) and Structured chart.
5. To perform the user's view analysis for the suggested system: Use case diagram.
6. Problem Analysis and Project Planning -Thorough study of the problem-Identify Project scope, objectives, and Infrastructure.
7. Software Requirement Analysis -Describe the individual Phases/modules of the project and identify.
8. Data Modelling -Use work products -data dictionary.
9. Software Designing -Develop use case diagrams and activity diagrams, build and test class diagrams, sequence diagrams and add interfaces to class diagrams.
10. Prototype model -Develop the prototype of the product.

Handwritten signatures and initials in blue ink.

Deep Learning Lab			
Course Code:	CS584	Course Credits:	2
Course Category:	PCC	Course (U / P):	P
Course Year (U / P):	1P	Course Semester (U / P):	2P
No. of Labs (Hrs/Week):	1 (4 HRS)	Mid Sem. Exam Hours:	-
Total No. of Labs:	10	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Provide hands-on experience with deep learning frameworks and tools.			
2. Implement and experiment with neural network models for real-world problems			
3. Understand training, optimisation, and evaluation of deep learning models.			
4. Apply deep learning techniques in domains such as computer vision, NLP, and time-series analysis.			
5. Encourage research-oriented thinking and reproducible experimentation.			
COURSE OUTCOMES			
At the end of the course the students should be able to:			
1. Implement basic to advanced neural network architectures using Python-based frameworks.			
2. Train, tune, and evaluate deep learning models on benchmark datasets.			
3. Apply CNNs, RNNs, and Transformer-based models to domain-specific problems.			
4. Analyze model performance using appropriate metrics and visualization tools			
5. Develop mini-projects and research prototypes using deep learning techniques.			

List of Experiments:**Software and Tools:**

- Programming Language: Python
- Frameworks/Libraries: TensorFlow, Keras, PyTorch
- Supporting Libraries: NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn, OpenCV, NLTK / SpaCy
- Platforms: Jupyter Notebook, Google Colab (optional)

List of Practical:

1. Python & DL Environment Setup: NumPy, Pandas, Matplotlib, TensorFlow/Keras installation, dataset preprocessing.
2. Single Layer Perceptron: Implementation from scratch and logical gate classification.
3. Multi-Layer Perceptron (MLP): Implementation from scratch and logical gate classification.
4. Backpropagation Algorithm: ANN model training and performance visualization.
5. Activation & Loss Functions: Comparison of Sigmoid, ReLU, Tanh, Softmax, MSE, Cross-Entropy.
6. Convolutional Neural Network (CNN): MNIST digit classification.
7. Image Classification using CNN: CIFAR-10 dataset with Dropout and Batch Normalization.
8. Sequence modeling or text classification using RNN.
9. Hyperparameter Tuning: Learning rate and optimizer comparison (SGD, Adam, RMSprop).
10. Recurrent Neural Network (RNN): Sequence prediction implementation.
11. LSTM Implementation: Time series forecasting and future value prediction.
12. Autoencoder: Dimensionality reduction and reconstruction analysis.
13. Transfer Learning: Fine-tuning pre-trained models (ResNet) on a custom dataset.

Handwritten signature: Anilank

36th BoS Date: 30-July-2026

Academic Batch: 2026 and Onwards

PROGRAM ELECTIVES

Bayan;

Software Reliability Engineering			
Course Code:	CS530	Course Credits:	3
Course Category:	PEC	Course (U / P)	P
Course Year (U / P):	1P	Course Semester (U / P):	2P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. To provide fundamental concepts of software reliability.			
2. To provide knowledge how to develop reliable software.			
3. To give knowledge on faults classification and fault tolerance attribute and systems.			
4. Discussion on the software and hardware fault tolerance with failure intensity functions.			
5. Discussion on concept of N-version programming.			
COURSE OUTCOMES			
At the end of the course, the students should be able to:			
1. Explain and classify the reliable and non reliable software.			
2. Differentiate the error, fault and failure during development of reliable software.			
3. Knowledge of fault classification and fault prevention.			
4. Classify various fault tolerance software and hardware.			
5. Knowledge of N-Version programming with code coverage and coding technique.			

UNIT I: SOFTWARE RELIABILITY

Measures of software reliability, Mean Time To Failure (MTTF), Mean Time Between Failure (MTBF), Mean Time To Recovery (MTTR), availability, maintainability, Musa's operational profiles and type-1 uncertainty, defect removal and type-2 uncertainty, reliability stability and reliability growth, hardware reliability vs. software reliability, failure probability density function and reliability function, Reliability prediction, reliability metrics.

UNIT II: DEVELOPMENT OF RELIABLE SOFTWARE

Reliable software, defect prevention, detection and removal, design for robustness, verification & validation, stabilization of requirements, design, code and test artefacts, active and passive fault detection, fault handling and correction, exceptions, survivability, reliability models, software availability model.

UNIT III: FAULT TOLERANCE IN HARDWARE SYSTEMS

Fault classification, fault tolerance attributes and system structure, fault prevention, anticipated and unanticipated fault, test generation for digital systems, combinational logic network, Boolean difference method, test generation for sequential circuits, fault simulation, application of hardware fault tolerance in developing fault tolerant software systems.

UNIT IV: SOFTWARE AND HARDWARE FAULT TOLERANCE

Software and hardware faults, failure intensity function, characterisation of fault injection, detection and correction, techniques for prediction of remaining faults and fault injection, classification tree analysis, code coverage, coding technique, fault tolerant & self-checking, fail-safe circuits, synchronous and asynchronous fail-safe circuits.

UNIT V: FAULT TOLERANT SOFTWARE

Concept of N-version programming (NVP) and methods, recovery block, acceptance tests, fault trees, validation of fault-tolerant systems, security, fault tolerance in wireless/mobile networks and Internet.

36th BoS Date: 30-July-2026

Academic Batch: 2026 and Onwards

Text Books/Reference Books:

1. Software Reliability Engineering, John D. Musa, Tata McGraw-Hill, 2005.
2. Fault-Tolerant Computer System Design, D.K. Pradhan, 2003.
3. Fault-Tolerant Computing, Theory & Techniques, D.K. Pradhan, Prentice Hall, 1986.
4. Reliable Computer Systems: Design and Evaluation, D. P. Siewiorek and R. S. Swartz, Digital Press, 1992.
5. Probability and Statistics with Reliability, Queueing and Computer Science applications, K.S.Trivedi, Prentice Hall, 1982.
6. Fault Tolerant Principles and Practice, Anderson and Lee, PHI, 1989.

Handwritten signatures in blue ink:
Helankar, Vaidyanathan, and two other illegible signatures.

Software Quality Assurance			
Course Code:	CS532	Course Credits:	3
Course Category:	PEC	Course (U / P)	P
Course Year (U / P):	1P	Course Semester (U / P):	2P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3

COURSE OBJECTIVES

1. Understand basic concepts of software quality and SQA principles.
2. Analyze quality factors and pre-project SQA activities.
3. Apply SQA techniques like reviews, V&V, and testing.
4. Evaluate software quality infrastructure and tools.
5. Assess quality metrics, standards, and management practices.

COURSE OUTCOMES

- At the end of the course, the students should be able to:
1. Understand basic software quality and SQA principles.
 2. Analyze software quality factors and SQA plans.
 3. Apply reviews, verification, validation, and testing techniques.
 4. Evaluate software quality infrastructure and tools.
 5. Assess quality metrics, standards, and management practices.

UNIT I: INTRODUCTION TO SOFTWARE QUALITY ASSURANCE

Software Quality Challenge, Uniqueness of Software Quality Assurance, Software Errors, Faults and Failures, Definition of Software Quality, Software Quality Assurance: Definition, Objectives and Scope, Relationship between Software Engineering and SQA, Software Quality Factors: Product Operation Factors, Product Revision Factors, Product Transition Factors, Software Quality Models, Software Quality Assurance vs Quality Control.

UNIT II: SQA SYSTEM AND PRE-PROJECT QUALITY COMPONENTS

Components of the Software Quality Assurance System, SQA Architecture and Framework, Pre-Project SQA Components, Contract Review Process, Objectives, Stages and Implementation, Development Plan and Quality Plan, Risk Management in Software Projects, SQA Considerations for Small and Internal Projects, Quality Factor Model.

UNIT III: SQA ACTIVITIES IN THE SOFTWARE DEVELOPMENT LIFE CYCLE

Integrating SQA into the Project Life Cycle, Verification and Qualification, Software Reviews: Formal Design Reviews, Peer Reviews and Inspections, Software Testing Strategies: White Box Testing, Black Box Testing, Test Planning and Test Case Design, Automated Testing, Alpha and Beta Testing Quality Assurance vs Testing.

UNIT IV: SOFTWARE QUALITY INFRASTRUCTURE COMPONENTS

Quality Assurance in Software Maintenance, Quality Assurance for External Participants and Outsourcing, CASE Tools and Their Impact on Software Quality, Procedures and Work Instructions, Supporting Quality Devices: Templates and Checklists, Staff Training and Certification, Corrective and Preventive Actions Software Configuration Management, Documentation Control.

36th BoS Date: 30-July-2026

Academic Batch: 2026 and Onwards

UNIT V: SOFTWARE QUALITY MANAGEMENT, METRICS AND STANDARDS

Project Progress Control, Software Quality Metrics: Process Metrics, Product Metrics, Cost of Software Quality, Quality Management Standards: ISO 9001 and ISO 9000-3, CMM, CMMI, SPICE, IEEE Software Engineering Standards, Role of Management in SQA, SQA Organization and SQA Units, Future Trends and Challenges in Software Quality Assurance

Text Books/Reference Books:

1. Daniel Galin, Software Quality Assurance: From Theory to Implementation, Pearson Education.
2. Software Quality Assurance by Jeff Tian, Wiley-IEEE Press, 2005.
3. Software Engineering by K. K. Agarwal & Yogesh Singh, New Age International Publishers, 2013.
4. Software Testing: Principles, Techniques and Tools by Yogesh Singh, Oxford University Press, 2008.

Handwritten signatures in blue ink.

Software Project Management			
Course Code:	CS534	Course Credits:	3
Course Category:	PEC	Course (U / P)	P
Course Year (U / P):	1P	Course Semester (U / P):	2P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. To provide fundamental skills of software Project management emphasising challenges.			
2. To provide project management concepts through working in a group as team leader.			
3. To give knowledge on different techniques of project monitoring, control and review.			
4. Discussion on the effect of project management practices in an organization.			
5. Discussion on Project monitoring, risk and quality control			
COURSE OUTCOMES			
At the end of the course, the students should be able to:			
1. Define the principles of project management for developing software.			
2. Explain various project management scheduling techniques.			
3. Classify various project management tools and estimate the risks involved in project activities.			
4. Assess issues related to project quality and staffing.			
5. Knowledge of different tools used in Software Project Management.			

UNIT I: SOFTWARE PROJECT MANAGEMENT, PLANNING AND ESTIMATION

Fundamentals of Software Project Management (SPM), Project Management Cycle, SPM Objectives, SPM Framework, Software Project Planning, Planning Objectives, Project Plan, Types of Project Plan, Structure of a Software Project Management Plan, Software Project Estimation, Estimation Methods, Estimation Models, Decision Process.

UNIT II: PROJECT ORGANIZATION AND SCHEDULING

Work Breakdown Structure (WBS), Types of WBS, Functions, Activities and Tasks, Project Life Cycle and Product Life Cycle, Ways to Organize Personnel, Project Schedule, Scheduling Objectives, Building the Project Schedule, Scheduling Terminology and Techniques, Network Diagrams: PERT, CPM, Bar Charts: Milestone Charts, Gantt Charts

UNIT III: PROJECT MONITORING AND CONTROL

Dimensions of Project Monitoring & Control, Earned Value Analysis, Earned Value Indicators: Budgeted Cost for Work Scheduled (BCWS), Cost Variance (CV), Schedule Variance (SV), Cost Performance Index (CPI), Schedule Performance Index (SPI), Software Reviews, Types of Review: Inspections, Deskchecks, Walkthroughs, Code Reviews

UNIT IV : SOFTWARE CONFIGURATION AND RISK MANAGEMENT

Software Configuration Items and Tasks, Baselines, Plan for Change, Change Control, Change Requests Management, Version Control, Risk Management: Risks and Risk Types, Risk Breakdown Structure (RBS), Risk Management Process: Risk Identification, Risk Analysis, Risk Planning, Risk Monitoring, Cost Benefit Analysis, Software Project Management Tools: CASE Tools, MS-Project

UNIT V: PROJECT QUALITY MANAGEMENT

Quality of project, Stages of software quality management, Software Quality Attributes, Software Quality Metrics and Indicators, Process vs Product Quality Management, Tools and techniques for quality control and management.

Text Books/Reference Books:

1. Software Project Management, Bob Hughes and Mike Cotterell, McGraw Hill
2. Information Technology Project Management, Kathy Schwalbe, International student edition, THOMSON course Technology, 2003.
3. Microsoft office Project 2003 Bible", Elaine Marmel, Wiley publishing Inc.
4. Software Project Management: A Unified Framework, Walker Royce, Addison-Wesley
5. A Practitioner's Guide to Software Engineering, Roger Pressman, Tata McGraw Hill, 2014 8th edition.
6. Basics of Software Project Management, NIIT, Prentice-Hall India, Latest Edition. Dan Woods, Open Source for the Enterprise: Managing Risks, Reaping Rewards, O'Reilly, 2005.

Handwritten signature in blue ink: "Anil Kumar" followed by a flourish.

Open Source Software Engineering			
Course Code:	CS536	Course Credits:	3
Course Category:	PEC	Course (U / P)	P
Course Year (U / P):	1P	Course Semester (U / P):	2P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Knowledge of open-source software.			
2. A General understanding of open-source technology.			
3. Understanding of open-source language.			
4. Understanding of open-source software applications and framework.			
5. Understanding of open-source in the enterprise.			
COURSE OUTCOMES			
At the end of the course, the students should be able to:			
1. Basic to advanced knowledge of open-source software.			
2. Ability to apply open-source technology.			
3. Ability to design, develop, maintain and evaluate open-source software application framework.			
4. Ability to apply open-source language.			
5. Ability to apply open-source software in the enterprise.			

UNIT I: OPEN-SOURCE SOFTWARE

Open-Source Software (OSS), history, philosophy and ethics of open-source software, open-source software development methodology, open source vs. closed source, open-source software vs. free software, open-source software vs. source available, Windows and Linux, open-source development environment, methods and models, standards, open-source standards, benefits of open standards, standard setting organizations and processes, project management via open source and open standard.

UNIT II: OPEN SOURCE TECHNOLOGY

Open source technology and platform, Operating system: Linux, Berkeley Software Distribution; web server: Apache; communication servers: send mail, jabber, application and messaging server: JBoss, Zope, Zend.

UNIT III: OPEN SOURCE LANGUAGES

Ruby, Ruby and object-orientation, data, expressions and flow control, class, object and modules, project and libraries, developing a basic Ruby application, PHP, configure environment, PHP basics, functions, arrays, object-oriented PHP, MYSQL, PostgreSQL

UNIT IV: OPEN SOURCE SOFTWARE APPLICATIONS AND FRAMEWORK

Open source desktop applications, Wiki software, LAMP application, web server and database server application, OSS management tools: taskjuggler, dotProject.net, rapid web application development framework: Ruby on Rails, Model-View-Controller model, Don't Repeat Yourself principle.

UNIT V: OPEN SOURCE SOFTWARE ENGINEERING

Nature of open source, leadership in open source software life cycle, designing an open source strategy, open source licenses, comparison of open source licenses, open source empowerment,

[Handwritten signatures and initials in blue ink]

requirements in an OSS project

Text Books/Reference Books

1. Paul Kavanagh, Open Source Software: Implementation and Management, Digital Press, 2004.
2. W. Jason Gilmore, Beginning PHP and MySQL, Apress, 2010.
3. Timothy Fisher, Ruby on Rails, Apress, 2009.
4. Dan Woods, Open Source for the Enterprise: Managing Risks, Reaping Rewards, O'Reilly, 2005.
5. James Lee, Brent Ware, Open Source Web Development with LAMP, Pearson Education, 2008.
6. Steven Weber, The Success of Open Source, Harvard University Press, 2004.
7. Peter Cooper, Beginning Ruby, Apress, 2007.

Helankar *Varadachari* *P*

Parallel and High-Performance Computing			
Course Code:	CS538	Course Credits:	3
Course Category:	PEC	Course (U / P)	P
Course Year (U / P):	1P	Course Semester (U / P):	2P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3

COURSE OBJECTIVES

1. Introduce the fundamental concepts of parallel and high-performance computing, including concurrency, parallelism, scalability, and performance limitations.
2. To develop the ability to analyse performance bottlenecks in parallel programs using profiling tools, memory hierarchy concepts, and efficiency metrics.
3. To enable students to identify parallelism in applications and design efficient parallel algorithms using CPU-based optimization and vectorization techniques.
4. To provide hands-on experience in developing and optimizing shared-memory parallel programs using the OpenMP programming model with appropriate scheduling and synchronization strategies.
5. To familiarize students with distributed-memory and accelerator-based computing by implementing parallel programs using MPI and GPU programming models and evaluating their scalability and performance.

COURSE OUTCOMES

At the end of the course, the students should be able to:

1. Explain the fundamental concepts of parallel and high-performance computing, including concurrency, scalability, and performance limitations.
2. Analyse performance bottlenecks in parallel programs using profiling techniques, memory hierarchy concepts, and efficiency metrics.
3. Design parallel solutions by identifying parallelism in applications and applying suitable parallel algorithms and CPU-based optimization techniques.
4. Develop and optimize shared-memory parallel programs using OpenMP, applying scheduling, synchronization, and vectorization strategies for improved performance.
5. Implement distributed and accelerator-based parallel programs using MPI and GPU programming models, and evaluate their scalability and performance.

UNIT I: INTRODUCTION TO PARALLEL AND HIGH-PERFORMANCE COMPUTING

Need for parallel computing, real-world applications of parallel and high-performance computing, concurrency vs parallelism, scalability basics, Amdahl's Law and motivation for high-performance computing

UNIT II: PERFORMANCE ANALYSIS

Performance bottlenecks in parallel programs, profiling techniques for performance analysis, performance limits and efficiency metrics, data layout and memory hierarchy, cache effects on program performance.

UNIT III: PARALLEL PROGRAM DESIGN AND CPU PARALLELISM

Planning parallel computing projects, identifying parallel opportunities in programs, parallel algorithms and common computation patterns, vectorisation concepts, SIMD architecture and speedup

Handwritten signatures and initials:
 [Signature] [Signature] [Signature]

UNIT IV: SHARED MEMORY PROGRAMMING

Loop transformations for performance optimisation, compiler support for vectorization, OpenMP programming model, OpenMP performance optimization techniques, scheduling, reduction, synchronisation overhead.

UNIT V: DISTRIBUTED MEMORY PARALLELISM AND GPU COMPUTING

Practical multicore performance tuning, need for MPI and distributed high-performance computing, MPI programming basics, point-to-point communication, collective communication operations, performance considerations in MPI, communication overhead, scalability, GPU architecture and computational advantages, GPU programming model, GPU execution concepts.

Text Books/Reference Books:

1. Parallel and High Performance Computing, Robert Robey and Yuliana Zamora, 2021.
 2. Introduction to High Performance Computing for Scientists and Engineers, Georg Hager, Gerhard Wellein, 2011.
 3. Introduction to parallel computing (2nd edition). Pearson Addison Wesley, 2003.
-   

SEMESTER III

Bayaan;

Cloud and Edge Computing			
Course Code:	CS621	Course Credits:	3
Course Category:	PCC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3

COURSE OBJECTIVES

1. Explain the principles and terminology of the foundations of cloud and edge computing.
2. Analyse problems and trade-offs concerning cloud architecture and virtualization.
3. Apply suitable methods drawn from distributed data and compute.
4. Design solutions that integrate edge-cloud continuum.
5. Evaluate approaches and responsible practice relating to security, reliability and operations.

COURSE OUTCOMES

At the end of the course, the students should be able to:

1. Explain the concepts and terminology of the foundations of Cloud and Edge Computing
2. Analyse problems using the methods covered under cloud architecture and virtualization.
3. Apply appropriate techniques from distributed data and compute to realistic datasets or systems.
4. Design a solution that integrates concepts from the edge-cloud continuum.
5. Evaluate alternative approaches using technical, performance and reliability criteria.
6. Develop and communicate a responsible, reproducible solution drawing on security, reliability and operations.

UNIT I: FOUNDATIONS OF CLOUD AND EDGE COMPUTING

Evolution from clusters and grids to cloud and edge computing; Definition and essential characteristics of cloud computing — on-demand self-service, broad network access, resource pooling, rapid elasticity and measured service; Overview of service models (IaaS, PaaS, SaaS) and deployment models (public, private, hybrid, multi-cloud); Drivers for edge computing — latency, bandwidth, privacy and data locality; The cloud-to-edge continuum at a glance; Representative cloud and edge use cases; Benefits, limitations and cost/performance trade-offs.

UNIT II: CLOUD ARCHITECTURE AND VIRTUALIZATION

IaaS, PaaS and SaaS; Virtual machines and containers; Orchestration; Service discovery; Autoscaling; Multi-tenancy; Cloud storage; Hypervisor types; Full and para-virtualization; Hardware support; Namespaces and control groups; Container images and registries; VM-container comparison; control loops; Resource isolation and scheduling; Public, private, hybrid and multi-cloud models; Regions and availability zones; Reference architectures; Elasticity policies; Resource provisioning; Load balancing; Cloud service selection; Performance, availability and cost comparison; Worked examples and case analysis.

UNIT III: DISTRIBUTED DATA AND COMPUTE

Distributed file systems; Key-value and NoSQL stores; Partitioning and replication; MapReduce, stream processing and serverless workflows; Data locality; Shuffling and partition strategies;

Signature

Distributed joins; Lineage and recomputation; Batch-stream unification; Data skew; Checkpointing; Back-pressure; Execution-plan analysis; Scalability experiments; Single-writer and multi-writer replication; Leader-follower and leaderless designs; Quorum reads and writes; Conflict-free replicated data types; Consistency anomalies; Isolation levels; Compensating actions; Failure recovery; Worked examples and case analysis; Comparative evaluation of techniques.

UNIT IV: EDGE-CLOUD CONTINUUM

Fog and edge architectures; Placement and offloading; Latency, bandwidth and energy trade-offs; IoT integration; Federated edge learning; Public, private, hybrid and multi-cloud models; Regions and availability zones; Reference architectures; Elasticity policies; Resource provisioning; Cloud service selection; Performance, availability and cost comparison; Edge node taxonomy; Cloudlets and gateways; Task-placement formulations; Computation offloading; Mobility support; Intermittent connectivity; Edge caching; Privacy-preserving analytics; Latency-energy modelling; Smart-city and industrial-IoT examples; Worked examples and case analysis.

UNIT V: SECURITY, RELIABILITY AND OPERATIONS

Zero-trust basics; Identity and access; Observability; SRE indicators; Resilience patterns; Cost governance; Sustainable computing; Case studies; Availability and reliability measures; Redundancy; Health checks; Graceful degradation; Bulkheads; Recovery-point and recovery-time objectives; Backup and disaster recovery; Fault injection; Chaos experiments; Reliability-cost trade-offs; Logs, metrics and traces; Service-level indicators, objectives and error budgets; Capacity planning; Deployment strategies; Runbooks; Incident response; Root-cause analysis; Post-incident reviews; Operational dashboards; Worked examples and case analysis.

Text Books/Reference Books:

1. M. van Steen and A. S. Tanenbaum, Distributed Systems, 4th ed., distributed-systems.net, 2023.
2. T. Erl, R. Puttini and Z. Mahmood, Cloud Computing: Concepts, Technology & Architecture, Pearson.
3. T. Hwang, G. Fox and J. Dongarra, Distributed and Cloud Computing, Morgan Kaufmann.
4. R. Buyya and A. V. Dastjerdi (eds.), Internet of Things: Principles and Paradigms, Morgan Kaufmann.
5. C. Pahl et al., Cloud Container Technologies: a State-of-the-Art Review, IEEE Transactions on Cloud Computing.

Handwritten signatures:
 Anant
 Anant
 Anant

AI Methods for Software Engineering			
Course Code:	CS623	Course Credits:	3
Course Category:	PCC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3

COURSE OBJECTIVES

1. Explain the principles and terminology of AI-enabled software engineering.
2. Analyse problems and trade-offs concerning code intelligence.
3. Apply suitable methods drawn from generative AI for development.
4. Design solutions that integrate AI for testing and operations.
5. Evaluate approaches and responsible practice relating to trustworthy adoption.

COURSE OUTCOMES

At the end of the course, the students should be able to:

1. Explain the fundamental concepts and terminology of AI-enabled software engineering.
2. Analyse problems using the methods covered under code intelligence.
3. Apply appropriate techniques from generative AI for development to realistic datasets or systems.
4. Design a solution that integrates concepts from AI for testing and operations.
5. Evaluate alternative approaches using technical, performance and reliability criteria.
6. Develop and communicate a responsible, reproducible solution drawing on trustworthy adoption.

UNIT I: AI-ENABLED SOFTWARE ENGINEERING

SE lifecycle data; Task formulation; Representations of code and requirements; Classical ML and deep learning foundations for software artifacts; Software-lifecycle tasks suitable for AI; Requirements, code, test and operations data; Supervised, self-supervised and generative formulations; Feature and representation choices; Dataset construction; Temporal validation; Human-in-the-loop workflows; Productivity and quality measures; Automation boundaries; Industrial adoption risks; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures.

UNIT II: CODE INTELLIGENCE

Code embeddings; Defect and vulnerability prediction; Clone detection; Code search; Program repair; Evaluation datasets and leakage; Token-, syntax-tree-, control-flow- and graph-based representations; Benchmark construction; Negative sampling; Top-k retrieval; Precision, recall, F1 and ranking measures; Temporal validation; Cross-project generalization; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises.

Arunkumar Jayaraman

UNIT III: GENERATIVE AI FOR DEVELOPMENT

Transformers and code LLMs; Prompting; Retrieval-augmented generation; Generation, review and refactoring; Agentic development workflows; Pretraining and fine-tuning objectives; Instruction tuning; Prompt patterns; Context-window management; Retrieval indexing; Grounding and citation; Decoding controls; Structured output; Tool calling; Automated and human evaluation; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs.

UNIT IV: AI FOR TESTING AND OPERATIONS

Test generation and prioritization; Log and trace analytics; Anomaly detection; Incident prediction; AIOps and human oversight; AI-based test generation; Test prioritization; Flaky-test prediction; Defect localization; Log parsing; Trace and metric correlation; Incident clustering; Root-cause assistance; Predictive maintenance of services; Alert-quality evaluation; Feedback loops; Safe automation and operator approval; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios.

UNIT V: TRUSTWORTHY ADOPTION

Quality metrics; Hallucination and security risks; Privacy and IP; Explainability; Responsible use; Empirical evaluation and industrial case studies; Data and model documentation; Robustness and distribution shift; Bias and fairness; Privacy; Intellectual-property considerations; Audit trails; Risk registers; Human approval gates; Post-deployment monitoring; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices.

Text / Reference Books:

1. M. Harman, S. A. Mansouri and Y. Zhang, Search-Based Software Engineering, ACM Computing Surveys.
2. C. C. Aggarwal, Machine Learning for Text, 2nd ed., Springer.
3. L. Allamanis et al., A Survey of Machine Learning for Big Code and Naturalness, ACM Computing Surveys.
4. M. Chen et al., Evaluating Large Language Models Trained on Code, arXiv, 2021.
5. C. Molnar, Interpretable Machine Learning, 2nd ed., 2022.

Handwritten signatures:
Aulanki
Vijay Kumar; P

AI-assisted Software Engineering Lab			
Course Code:	CS681	Course Credits:	2
Course Category:	PCC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Labs (Hrs/Week):	1 (4 HRS)	Mid Sem. Exam Hours:	-
Total No. of Labs:	10	End Sem. Exam Hours:	3

COURSE OBJECTIVES

1. Configure and use contemporary tools required for the laboratory domain.
2. Implement core algorithms and system components through reproducible experiments.
3. Measure correctness, performance, robustness and resource trade-offs.
4. Integrate multiple components into a working domain-specific solution.
5. Document, demonstrate and critically evaluate responsible engineering practice.

COURSE OUTCOMES

At the end of the course, the students should be able to:

1. Set up and operate the required laboratory platforms and development environment.
2. Implement and test core techniques using appropriate datasets, simulators or systems.
3. Collect and analyse experimental evidence using suitable metrics.
4. Diagnose failures and improve a solution through systematic experimentation.
5. Develop an integrated mini-project with reproducible code and documentation.

PRACTICALS

Note: Each practical shall include objective, design/algorithm, implementation, test data, observations, results and a brief critical analysis. Faculty may adapt platforms while retaining the stated learning outcomes.

Lab 1: Mining and Preparing Software Data

1. Extract commits, issues and code-review data from a repository; construct a documented dataset.
2. Build code/text representations and assess missingness, imbalance and leakage.
3. Create a reproducible experiment pipeline with versioned data and configurations.

Lab 2: Defect and Vulnerability Prediction

1. Train baseline and ensemble defect predictors.
2. Evaluate temporal splits, calibration and cost-sensitive metrics.
3. Explain predictions and inspect false positives with developers' perspective.

Lab 3: Code Search and Classification

1. Build semantic code search using embeddings.
2. Classify issues or commits and compare classical and transformer models.
3. Evaluate retrieval with precision, recall, MRR and qualitative error analysis.

Lab 4: LLM-assisted Requirements

1. Design prompts to identify ambiguity and generate acceptance criteria.
2. Ground outputs in a supplied requirements repository.
3. Measure factuality, usefulness and reviewer agreement.

Handwritten signatures and initials are present at the bottom of the page.

Lab 5: Code Generation and Review

1. Generate a bounded function with tests using a code model.
2. Perform automated and human review for correctness, security and maintainability.
3. Compare prompt and retrieval strategies.

Lab 6: Automated Test Generation

1. Generate unit and property-based tests.
2. Measure statement, branch and mutation coverage.
3. Analyze equivalent mutants and weak oracles.

Lab 7: Program Repair and Refactoring

1. Localize faults from failing tests.
2. Generate and validate candidate patches.
3. Refactor a code smell and verify behavioural equivalence.

Lab 8: AIOps Analytics

1. Parse logs and traces; derive operational features.
2. Detect anomalies and cluster incidents.
3. Create an explainable incident summary and assess alert quality.

Lab 9: Responsible AI-SE Evaluation

1. Probe data leakage, hallucination and insecure suggestions.
2. Prepare a model/system card and mitigation plan.
3. Conduct a controlled comparison with statistical reporting.

Lab 10: Integrated Mini-project

1. Build an AI-assisted SE tool or workflow.
2. Package it with tests, documentation and reproducible evaluation.
3. Demonstrate outcomes and defend design, limitations and ethical safeguards.

Handwritten signatures:
Aelanki
Vijay Kumar; P

PROGRAM ELECTIVES

Barbieri

P

Advanced Software Architecture and Design			
Course Code:	CS631	Course Credits:	3
Course Category:	PEC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Explain the principles and terminology of architectural foundations.			
2. Analyse problems and trade-offs concerning architectural styles.			
3. Apply suitable methods drawn from domain and component design.			
4. Design solutions that integrate architecture evaluation and evolution.			
5. Evaluate approaches and responsible practice relating to contemporary architectures.			
COURSE OUTCOMES			
At the end of the course, the students should be able to:			
1. Explain the fundamental concepts and terminology of architectural foundations.			
2. Analyse problems using the methods covered under architectural styles.			
3. Apply appropriate techniques from domain and component design to realistic datasets or systems.			
4. Design a solution that integrates concepts from architecture evaluation and evolution.			
5. Evaluate alternative approaches using technical, performance and reliability criteria.			
6. Develop and communicate a responsible, reproducible solution drawing on contemporary architectures.			

UNIT I: ARCHITECTURAL FOUNDATIONS

Quality attributes; Architectural drivers; Viewpoints; Documentation; Architecture decision records; Technical debt; Stakeholder concerns; Functional and quality-attribute requirements; Scenarios and utility trees; Views and viewpoints; Component-and-connector models; Deployment views; Constraints; Risks and technical debt; Problem formulation; Terminology and assumptions; Mathematical representation; Baseline methods; Worked numerical examples; Algorithmic steps; Computational complexity; Evaluation measures; Practical limitations and application contexts; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures.

UNIT II: ARCHITECTURAL STYLES

Layered, event-driven, pipe-and-filter, microkernel, service-oriented and data-centric styles; Trade-off analysis; Stakeholder concerns; Functional and quality-attribute requirements; Scenarios and utility trees; Views and viewpoints; Component-and-connector models; Deployment views; Architecture decision records; Constraints; Risks and technical debt; Applicability, structure, benefits and liabilities of each style; Hybrid architectures; Coupling and cohesion; Control and data flow; Scalability and modifiability analysis; Style-selection exercises using enterprise case problems; Worked examples and case analysis; Comparative evaluation of techniques.

Handwritten signatures and initials in blue ink.

UNIT III: DOMAIN AND COMPONENT DESIGN

Domain-driven design; Bounded contexts; Components and connectors; APIs; Contracts; Dependency management; Ubiquitous language; Entities, value objects and aggregates; Context mapping; Domain events; Component responsibilities; Provided and required interfaces; Dependency inversion; Cohesion and coupling; API contracts; Package and deployment structure; Design-pattern selection; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices.

UNIT IV: ARCHITECTURE EVALUATION AND EVOLUTION

ATAM; Scenario-based evaluation; Fitness functions; Refactoring; Modernization; Product-line architecture; Quality-attribute scenarios; Utility trees; Architecture Tradeoff Analysis Method; Sensitivity and trade-off points; Risk themes; Architecture fitness functions; Technical-debt assessment; Evolutionary architecture; Refactoring strategies; Strangler migration; Modernization roadmaps; Architecture governance and decision review; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices.

UNIT V: CONTEMPORARY ARCHITECTURES

Cloud-native, reactive and serverless systems; Resilience; Observability; Security architecture; Case-study design review; Reactive manifesto; Cloud-native reference models; Function-as-a-service; Event streaming; Data-intensive architecture; Evolutionary architecture; Architecture fitness functions; Platform engineering; Modernization and migration patterns; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies; Scalability considerations.

Text Books/Reference Books:

1. L. Bass, P. Clements and R. Kazman, Software Architecture in Practice, 4th ed., Addison-Wesley.
2. N. Ford et al., Software Architecture: The Hard Parts, O'Reilly.
3. M. Richards and N. Ford, Fundamentals of Software Architecture, 2nd ed., O'Reilly.
4. E. Evans, Domain-Driven Design, Addison-Wesley.
5. R. N. Taylor, N. Medvidovic and E. Dashofy, Software Architecture, Wiley.

Cloud-Native and Microservices Engineering			
Course Code:	CS633	Course Credits:	3
Course Category:	PEC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3

COURSE OBJECTIVES

1. Explain the principles and terminology of cloud-native principles.
2. Analyse problems and trade-offs concerning microservice design.
3. Apply suitable methods drawn from distributed data patterns.
4. Design solutions that integrate deployment and operations.
5. Evaluate approaches and responsible practice relating to resilience and governance.

COURSE OUTCOMES

At the end of the course, the students should be able to:

1. Explain the fundamental concepts and terminology of cloud-native principles.
2. Analyse problems using the methods covered under microservice design.
3. Apply appropriate techniques from distributed data patterns to realistic datasets or systems.
4. Design a solution that integrates concepts from deployment and operations.
5. Evaluate alternative approaches using technical, performance and reliability criteria.
6. Develop and communicate a responsible, reproducible solution drawing on resilience and governance.

UNIT I: CLOUD-NATIVE PRINCIPLES

Twelve-factor applications; Containers; Immutable infrastructure; API-first design; Decomposition criteria; Cloud-native maturity; Application of the twelve-factor methodology; Stateless process design; Externalized configuration; Backing services; Disposability; Horizontal scaling; Container image layers; Immutable deployment; API-first contracts; ; Decomposition by business capability; Cloud-native readiness assessment; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices.

UNIT II: MICROSERVICE DESIGN

Bounded contexts; Synchronous and asynchronous communication; API gateway; Service mesh; Data ownership; Service-boundary identification; Granularity trade-offs; Synchronous REST and gRPC; Asynchronous messaging; Backend-for-frontend; Service discovery; Database-per-service; Contract versioning; Consumer-driven contract testing; Inter-service authentication; Anti-corruption layers; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies.

Handwritten signatures:
 1. A signature that appears to read "Arun" or "Aruno".
 2. A signature that appears to read "Rajesh".
 3. A circular stamp or signature on the right.

UNIT III: DISTRIBUTED DATA PATTERNS

Sagas; CQRS; Event sourcing; Idempotency; Eventual consistency; Transactional outbox; Database-per-service; Saga choreography; Compensating transactions; Change-data capture; Idempotent consumers; Deduplication; Ordering guarantees; Read-model construction; Conflict handling; Schema evolution; Failure and recovery scenarios; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies; Scalability considerations.

UNIT IV: DEPLOYMENT AND OPERATIONS

Kubernetes primitives; Configuration; CI/CD; Progressive delivery; Telemetry; SLOs; Autoscaling; Kubernetes pods, deployments, services and ingress; Configuration maps and secrets; Health probes; Resource requests and limits; Horizontal autoscaling; Rolling, blue-green and canary delivery; GitOps; CI/CD quality gates; Distributed tracing; Metrics and logs; Service-level indicators and objectives; Rollback and incident response; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios.

UNIT V: RESILIENCE AND GOVERNANCE

Timeouts, retries and circuit breakers; Chaos engineering; Security; Policy; Cost and sustainability; Migration case study and bulkheads; Health checking; Graceful degradation; Chaos testing; Service ownership; Platform standards; API governance; Compliance controls; Cost allocation; Sustainability indicators; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs.

Text Books/Reference Books:

1. S. Newman, Building Microservices, 2nd ed., O'Reilly.
 2. C. Richardson, Microservices Patterns, Manning.
 3. B. Burns, Designing Distributed Systems, 2nd ed., O'Reilly.
 4. K. Morris, Infrastructure as Code, 2nd ed., O'Reilly.
 5. B. Beyer et al., Site Reliability Engineering, O'Reilly.
- Handwritten signatures: Helankar, Jayaram, P*

Advanced Software Testing			
Course Code:	CS635	Course Credits:	3
Course Category:	PEC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3

COURSE OBJECTIVES

1. Explain the principles and terminology of testing foundations.
2. Analyse problems and trade-offs concerning model- and property-based testing.
3. Apply suitable methods drawn from static and dynamic analysis.
4. Design solutions that integrate formal verification.
5. Evaluate approaches and responsible practice relating to verification in practice.

COURSE OUTCOMES

At the end of the course, the students should be able to:

1. Explain the fundamental concepts and terminology of testing foundations.
2. Analyse problems using the methods covered under model- and property-based testing.
3. Apply appropriate techniques from static and dynamic analysis to realistic datasets or systems.
4. Design a solution that integrates concepts from formal verification.
5. Evaluate alternative approaches using technical, performance and reliability criteria.
6. Develop and communicate a responsible, reproducible solution drawing on verification in practice.

UNIT I: TESTING FOUNDATIONS

Test adequacy; Coverage criteria; Mutation testing; Fault models; Combinatorial testing; Problem formulation; Terminology and assumptions; Mathematical representation; Baseline methods; Worked numerical examples; Algorithmic steps; Computational complexity; Evaluation measures; Practical limitations and application contexts; Test levels and testability; Specification- and structure-based design; Boundary and equivalence analysis; Fault seeding; Mutation score; Test prioritization; Flaky-test management; Regression selection; Test-oracle limitations; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations.

UNIT II: MODEL- AND PROPERTY-BASED TESTING

State models; Contracts; Generators; Oracles; Metamorphic testing; Differential testing; Test levels and testability; Specification- and structure-based design; Boundary and equivalence analysis; Coverage criteria; Fault seeding; Mutation score; Test prioritization; Flaky-test management; Regression selection; Test-oracle limitations; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies.

Arulank

Sanjay

UNIT III: STATIC AND DYNAMIC ANALYSIS

Data-flow analysis; Symbolic execution; Concolic testing; Fuzzing; Runtime verification; Control-flow graphs; Reaching definitions; Live-variable and taint analysis; Abstract interpretation; Path constraints; Symbolic states; Solver interaction; Dynamic instrumentation; Coverage-guided fuzzing; False-positive management; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies; Scalability considerations; Ethical and security implications.

UNIT IV: FORMAL VERIFICATION

Propositional and temporal logic; Model checking; Theorem proving; SAT/SMT; Program invariants; State-transition systems; Safety and liveness; Invariants; Reachability; Temporal operators; Kripke structures; State-space explosion; Abstraction; Refinement; Counterexample analysis; Bounded and probabilistic verification; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies; Scalability considerations.

UNIT V: VERIFICATION IN PRACTICE

Testing concurrent and distributed systems; Safety/security cases; CI integration; Evaluation of tools and evidence; Verification plans; Assurance cases; Concurrent and distributed-system properties; Race and deadlock detection; Security-property verification; Abstraction boundaries; Tool qualification; Evidence traceability; Industrial limitations; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies.

Text Books/ Reference Books:

1. P. Ammann and J. Offutt, Introduction to Software Testing, 2nd ed., Cambridge.
2. G. J. Myers, C. Sandler and T. Badgett, The Art of Software Testing, 3rd ed., Wiley.
3. C. Baier and J.-P. Katoen, Principles of Model Checking, MIT Press.
4. D. Peled, Software Reliability Methods, Springer.
5. E. M. Clarke, O. Grumberg and D. Peled, Model Checking, MIT Press.

Handwritten signatures:
Aelanko
P
B. S. S. S.

Software Measurement and Estimation			
Course Code:	CS637	Course Credits:	3
Course Category:	PEC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3

COURSE OBJECTIVES

1. Understand the principles of software measurement and estimation.
2. Apply software size, effort, cost, and schedule estimation techniques.
3. Analyse software quality and productivity metrics for project management.
4. Evaluate estimation models and risk factors in software development.
5. Develop accurate estimation strategies for real-world software projects.

COURSE OUTCOMES

At the end of the course, the students should be able to:

1. Explain software measurement concepts and metrics.
2. Apply software size and effort estimation techniques.
3. Analyse software quality, productivity, and project metrics.
4. Evaluate estimation models for software projects.
5. Develop estimation plans for software development projects.
6. Assess software project performance using measurement and estimation techniques.

UNIT I: INTRODUCTION TO SOFTWARE MEASUREMENT

Software measurement fundamentals; Goals of software measurement; Measurement theory; Scales of measurement; Direct and indirect measures; Software metrics; Product, process and project metrics; Goal Question Metric (GQM); Measurement planning; Software quality attributes; Measurement repositories; ISO/IEC software measurement standards.

UNIT II: SOFTWARE SIZE ESTIMATION

Software size estimation; Lines of Code (LOC); Function Point Analysis (FPA); Feature Points; Use Case Points; Object Points; COSMIC Function Points; Story Points in Agile; Requirement-based estimation; Early size estimation; Software complexity measures; Halstead metrics; Cyclomatic complexity.

UNIT III: SOFTWARE EFFORT AND COST ESTIMATION

Software effort estimation; Cost estimation principles; COCOMO and COCOMO II; Putnam Model; SLIM Model; Delphi estimation; Expert judgment; Wideband Delphi; Three-point estimation; PERT; Work Breakdown Structure (WBS); Resource estimation; Schedule estimation; Risk-adjusted estimation; Agile estimation techniques.

UNIT IV: SOFTWARE QUALITY AND PROJECT METRICS

Software quality metrics; Reliability metrics; Maintainability metrics; Defect density; Defect prediction; Productivity metrics; Earned Value Management (EVM); Schedule variance; Cost variance; Software dashboards; KPI measurement; Benchmarking; Process capability; Measurement for DevOps and Agile projects.

[Handwritten signatures and initials]

NIT V: ADVANCED ESTIMATION AND AI-DRIVEN SOFTWARE ANALYTICS

Machine Learning for software estimation; AI-based effort prediction; Predictive analytics for software projects; Estimation using historical repositories; Software analytics; Mining software repositories; Technical debt measurement; Risk analytics; Estimation tools; Continuous measurement; DevOps metrics; Future trends in software measurement and estimation; Industry case studies.

Text Books/ Reference Books:

1. Norman E. Fenton and James Bieman, Software Metrics: A Rigorous and Practical Approach, CRC Press.
2. Capers Jones, Applied Software Measurement, McGraw-Hill.
3. Steve McConnell, Software Estimation: Demystifying the Black Art, Microsoft Press.
4. Barry W. Boehm et al., Software Cost Estimation with COCOMO II, Pearson.
5. Roger S. Pressman and Bruce Maxim, Software Engineering: A Practitioner's Approach, McGraw-Hill.
6. Ian Sommerville, Software Engineering, Pearson.
7. Watts S. Humphrey, Managing the Software Process, Addison-Wesley.

Handwritten signature

Handwritten signature

Handwritten signature

UNIT V: ADVANCED ESTIMATION AND AI-DRIVEN SOFTWARE ANALYTICS

Machine Learning for software estimation; AI-based effort prediction; Predictive analytics for software projects; Estimation using historical repositories; Software analytics; Mining software repositories; Technical debt measurement; Risk analytics; Estimation tools; Continuous measurement; DevOps metrics; Future trends in software measurement and estimation; Industry case studies.

Text Books/ Reference Books:

1. Norman E. Fenton and James Bieman, Software Metrics: A Rigorous and Practical Approach, CRC Press.
2. Capers Jones, Applied Software Measurement, McGraw-Hill.
3. Steve McConnell, Software Estimation: Demystifying the Black Art, Microsoft Press.
4. Barry W. Boehm et al., Software Cost Estimation with COCOMO II, Pearson.
5. Roger S. Pressman and Bruce Maxim, Software Engineering: A Practitioner's Approach, McGraw-Hill.
6. Ian Sommerville, Software Engineering, Pearson.
7. Watts S. Humphrey, Managing the Software Process, Addison-Wesley.

Helanki

Varun;

R

Secure Software Engineering and DevSecOps			
Course Code:	CS639	Course Credits:	3
Course Category:	PEC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Explain the principles and terminology of the secure development lifecycle.			
2. Analyse problems and trade-offs concerning threat modelling and design.			
3. Apply suitable methods drawn from secure implementation and testing.			
4. Design solutions that integrate DevSecOps pipeline.			
5. Evaluate approaches and responsible practice relating to operations and assurance.			
COURSE OUTCOMES			
At the end of the course, the students should be able to:			
1. Explain the fundamental concepts and terminology of the secure development lifecycle.			
2. Analyse problems using the methods covered under threat modelling and design.			
3. Apply appropriate techniques from secure implementation and testing to realistic datasets or systems.			
4. Design a solution that integrates concepts from the DevSecOps pipeline.			
5. Evaluate alternative approaches using technical, performance and reliability criteria.			
6. Develop and communicate a responsible, reproducible solution drawing on operations and assurance.			

UNIT I: SECURE DEVELOPMENT LIFECYCLE

Threat landscape; Security requirements; Misuse cases; Risk assessment; Secure design principles; Security governance; Asset classification; Abuse and misuse cases; Risk rating; Secure design review; Coding standards; Verification gates; Release approval; Vulnerability disclosure and response; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies.

UNIT II: THREAT MODELLING AND DESIGN

Assets and trust boundaries; STRIDE; Attack trees; Secure architecture patterns; Privacy by design; System decomposition; Data-flow diagrams; Attacker goals; STRIDE and related taxonomies; Risk prioritization; Mitigations; Residual risk; Threat-model maintenance; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies; Scalability considerations; Ethical and security implications.

UNIT III: SECURE IMPLEMENTATION AND TESTING

Common weaknesses; Secure coding; SAST, DAST, IAST; Dependency and secret scanning; Fuzzing; Input validation; Output encoding; Authentication and session controls; Access-control

Handwritten signature: Helankar

checks; Secure error handling; Cryptographic API use; SAST, DAST and IAST; Dependency scanning; Secret detection; Fuzz testing; Security unit tests; Vulnerability triage; Remediation verification; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices.

UNIT IV: DevSecOps PIPELINE

Policy as code; Software composition analysis; SBOM; Artifact signing; Supply-chain security; Container and IaC scanning; Shift-left and continuous assurance; SAST, DAST, IAST and SCA placement; Secrets detection; SBOM generation; Signed artefacts; Provenance; Container and infrastructure scanning; Security gates and exceptions; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs.

UNIT V: OPERATIONS AND ASSURANCE

Zero trust; Monitoring and incident response; Vulnerability management; Compliance evidence; Maturity metrics; Case studies; Runtime protection; Identity and access review; Security telemetry; SIEM integration; Vulnerability and patch management; Incident detection and response; Forensic readiness; Penetration testing; Red-team exercises; Audit trails; Security metrics; Post-incident improvement; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices.

Text Books/ Reference Books:

1. G. McGraw, Software Security: Building Security In, Addison-Wesley.
2. A. Shostack, Threat Modeling, Wiley.
3. M. Howard and S. Lipner, The Security Development Lifecycle, Microsoft Press.
4. NIST, Secure Software Development Framework (SSDF), SP 800-218.
5. OWASP, Application Security Verification Standard and Testing Guide.

Handwritten signatures:
Anil Kumar
Vijay Kumar

Empirical Software Engineering			
Course Code:	CS641	Course Credits:	3
Course Category:	PEC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Explain the principles and terminology of research foundations.			
2. Analyse problems and trade-offs concerning quantitative study design.			
3. Apply suitable methods drawn from qualitative and mixed methods.			
4. Design solutions that integrate analysis and reporting.			
5. Evaluate approaches and responsible practice relating to evidence synthesis.			
COURSE OUTCOMES			
At the end of the course, the students should be able to:			
1. Explain the fundamental concepts and terminology of research foundations.			
2. Analyse problems using the methods covered under quantitative study design.			
3. Apply appropriate techniques from qualitative and mixed methods to realistic datasets or systems.			
4. Design a solution that integrates concepts from analysis and reporting.			
5. Evaluate alternative approaches using technical, performance and reliability criteria.			
6. Develop and communicate a responsible, reproducible solution drawing on evidence synthesis.			

UNIT I: RESEARCH FOUNDATIONS

Empirical questions; Theory and constructs; Validity; Ethics; Reproducibility; Open science; Problem formulation; Terminology and assumptions; Mathematical representation; Baseline methods; Worked numerical examples; Algorithmic steps; Computational complexity; Evaluation measures; Practical limitations and application contexts; Research problem, questions and hypotheses; Constructs and operational definitions; Literature positioning; Units of analysis; Sampling frame; Instrumentation; Threats to validity; Ethics approval; Preregistration; Replication package; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations.

UNIT II: QUANTITATIVE STUDY DESIGN

Controlled experiments; Quasi-experiments; Surveys; Sampling; Measurement; Power and effect size; Descriptive measures; Probability distributions; Confidence intervals; Effect sizes; Null-hypothesis tests; Multiple-comparison correction; Correlation and regression; Power analysis; Assumption checking; Robust alternatives; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies; Scalability considerations; Ethical and security implications.

Helankar Jayaram

UNIT III: QUALITATIVE AND MIXED METHODS

Interviews; Case studies; Grounded theory; Thematic analysis; Triangulation; Mixed-method designs; Interview and observation protocols; Purposive sampling; Transcription; Open, axial and selective coding; Memoing; Saturation; Inter-coder agreement; Member checking; Reflexivity; Reporting with evidence; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies; Scalability considerations.

UNIT IV: ANALYSIS AND REPORTING

Descriptive and inferential statistics; Regression; Non-parametric tests; Qualitative coding; Uncertainty and visualization; Data cleaning; Exploratory visualisation; Assumption checking; Effect sizes and confidence intervals; Regression and non-parametric methods; Mixed-method integration; Uncertainty; Transparent tables and figures; Reproducible scripts; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies.

UNIT V: EVIDENCE SYNTHESIS

Systematic literature reviews; Mapping studies; Meta-analysis; Replication; Artifact evaluation; Research-paper critique; Review protocol; Database and search-string design; Inclusion and exclusion criteria; Quality appraisal; Study coding; PRISMA-style flow; Narrative synthesis; Heterogeneity; Publication bias; Research-gap formulation; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies.

Text Books/ Reference Books:

1. C. Wohlin et al., Experimentation in Software Engineering, Springer.
2. R. K. Yin, Case Study Research and Applications, 6th ed., Sage.
3. J. W. Creswell and J. D. Creswell, Research Design, 6th ed., Sage.
4. B. Kitchenham, D. Budgen and P. Brereton, Evidence-Based Software Engineering and Systematic Reviews, CRC.
5. A. Field, Discovering Statistics Using R, Sage.

Handwritten signatures: Helankar, P. S. Rao, and a circled 'P'.

Software Engineering Analytics			
Course Code:	CS643	Course Credits:	3
Course Category:	PEC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Explain the principles and terminology of software repository ecosystems.			
2. Analyse problems and trade-offs concerning data extraction and integration.			
3. Apply suitable methods drawn from repository analytics.			
4. Design solutions that integrate text and code mining.			
5. Evaluate approaches and responsible practice relating to evaluation and deployment.			
COURSE OUTCOMES			
At the end of the course, the students should be able to:			
1. Explain the fundamental concepts and terminology of software repository ecosystems.			
2. Analyse problems using the methods covered under data extraction and integration.			
3. Apply appropriate techniques from repository analytics to realistic datasets or systems.			
4. Design a solution that integrates concepts from text and code mining.			
5. Evaluate alternative approaches using technical, performance and reliability criteria.			
6. Develop and communicate a responsible, reproducible solution drawing on evaluation and deployment.			

UNIT I: SOFTWARE REPOSITORY ECOSYSTEMS

Version control, issue trackers, code review, CI and telemetry; Research questions; Data provenance; Commit histories; Branches and merges; Pull requests; Issues; Continuous-integration logs; Releases and dependencies; API-rate limits; Bot filtering; Identity merging; Time-zone normalization; Ethical handling of developer data; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs.

UNIT II: DATA EXTRACTION AND INTEGRATION

APIs; Commit and change reconstruction; Identity resolution; Linking artifacts; Data quality and privacy; Repository APIs; Pagination and rate limits; Incremental harvesting; Schema mapping; Artefact linkage; Timestamp normalization; Duplicate and bot detection; Provenance; Privacy-preserving storage; Quality checks; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies.

UNIT III: REPOSITORY ANALYTICS

Descriptive metrics; Defect prediction; Ownership and socio-technical networks; Process and productivity analytics; Commit histories; Branches and merges; Pull requests; Issues; Continuous-integration logs; Releases and dependencies; API-rate limits; Bot filtering; Identity merging; Time-zone normalization; Ethical handling of developer data; Descriptive, diagnostic, predictive and

Handwritten signatures:
 Anand
 Jayaram

prescriptive questions; Metric validation; Temporal and project-wise splits; Class imbalance; Concept drift; Actionable visualisation; Uncertainty communication; Stakeholder-oriented interpretation; Worked examples and case analysis; Comparative evaluation of techniques.

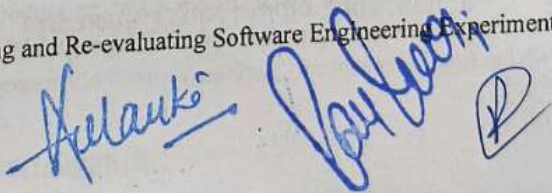
UNIT IV: TEXT AND CODE MINING

Topic models and embeddings; Issue classification; Sentiment pitfalls; Code representations; Clone and change analysis; Tokenization and parsing; AST and graph extraction; Bag-of-words and embeddings; Topic modelling; Classification and retrieval; Issue-commit linking; Evaluation datasets; Leakage and interpretability; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies.

UNIT V: EVALUATION AND DEPLOYMENT

Temporal validation; Imbalance and leakage; Explainability; Dashboards; Actionable insights; Responsible analytics; Temporal and cross-project validation; Benchmark and baseline selection; Class imbalance; Leakage prevention; Calibration; Interpretability; Statistical and practical significance; Dashboard design; Stakeholder actionability; Model packaging; Batch and online inference; Drift monitoring; Retraining triggers; Privacy and governance; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices.

Text Books / Reference Books:

1. T. Menzies, L. Williams and T. Zimmermann (eds.), Perspectives on Data Science for Software Engineering, Morgan Kaufmann.
 2. A. E. Hassan, The Road Ahead for Mining Software Repositories, FoSE.
 3. C. Bird, T. Menzies and T. Zimmermann (eds.), The Art and Science of Analyzing Software Data, Morgan Kaufmann.
 4. I. H. Witten et al., Data Mining: Practical Machine Learning Tools and Techniques, 4th ed., Morgan Kaufmann.
 5. M. D. Syer et al., Replicating and Re-evaluating Software Engineering Experiments, IEEE Software.
- 

Software Product Management and Requirements Analytics			
Course Code:	CS645	Course Credits:	3
Course Category:	PEC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3

COURSE OBJECTIVES

1. Explain the principles and terminology of product strategy.
2. Analyse problems and trade-offs concerning requirements engineering.
3. Apply suitable methods drawn from product discovery and experimentation.
4. Design solutions that integrate requirements analytics.
5. Evaluate approaches and responsible practice relating to delivery and governance.

COURSE OUTCOMES

At the end of the course, the students should be able to:

1. Explain the fundamental concepts and terminology of product strategy.
2. Analyse problems using the methods covered under requirements engineering.
3. Apply appropriate techniques from product discovery and experimentation to realistic datasets or systems.
4. Design a solution that integrates concepts from requirements analytics.
5. Evaluate alternative approaches using technical, performance and reliability criteria.
6. Develop and communicate a responsible, reproducible solution drawing on delivery and governance.

UNIT I: PRODUCT STRATEGY

Product management fundamentals; Product lifecycle; Product vision; Product strategy; Business models; Customer value proposition; Market segmentation; Competitive analysis; Stakeholder analysis; Product positioning; Opportunity assessment; Prioritization frameworks; Outcome-based roadmaps; Release planning; Product portfolio management; Product success metrics; Product lifecycle decisions; Product risk management; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Contemporary tools and practices.

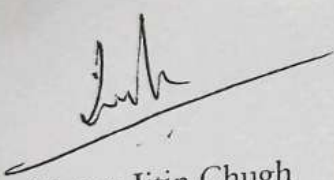
UNIT II: REQUIREMENTS ENGINEERING

Requirements engineering process; Requirements elicitation; Stakeholder identification; Interviews; Workshops; Observation techniques; User stories; Use cases; Functional requirements; Non-functional requirements; Quality attributes; Acceptance criteria; Requirements prioritization; Traceability; Validation and verification; Requirement baselining; Requirement change management; Change-impact analysis; Requirements documentation; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Contemporary tools and practices.

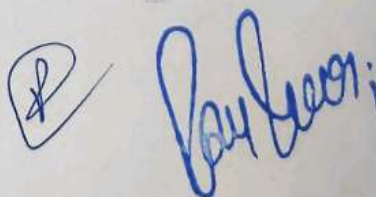
INTERNSHIP CERTIFICATE

This is to certify that Rishu Chaudhary, pursuing his B.Tech, 4th year at Gautam Buddha University has successfully completed summer internship at S-cube Storage Systems Pvt Ltd on C sharp from May 19th to July 20th

We found her sincere, hardworking and result oriented. She worked well as part of a team during her tenure. We take this opportunity to thank her and wish her all the best for her future.


Name: Jitin Chugh
Designation: Director
Place: Netaji Shubhash Place
Date: 27 July 2015
Office Stamp:

S-Cube Storage Systems Pvt. Ltd.
106, Best Sky Tower, Netaji Subhash
Place, Pitampura, Delhi-110034



UNIT III: PRODUCT DISCOVERY AND EXPERIMENTATION

Design thinking; Customer discovery; Problem validation; Hypothesis formulation; Lean Startup; MVP design; Rapid prototyping; Wireframing; Usability testing; A/B testing; Multivariate testing; Experiment design; Randomization; Sample size estimation; Statistical significance; Guardrail metrics; Funnel analysis; Cohort analysis; Product analytics; Experiment evaluation; Data-driven decision making; Feature validation; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Contemporary tools and practices.

UNIT IV: REQUIREMENTS ANALYTICS

Requirements mining; Feedback mining; Review mining; NLP for requirements; Text preprocessing; Topic modelling; Requirement classification; Requirement clustering; Duplicate requirement detection; Requirement similarity analysis; Sentiment analysis; Ambiguity detection; Requirement quality assessment; AI-based prioritization; Predictive analytics; Requirement traceability analytics; Visualization dashboards; Descriptive, diagnostic, predictive and prescriptive analytics; Metric validation; Concept drift; Stakeholder-oriented interpretation; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Contemporary AI tools and practices.

UNIT V: DELIVERY AND GOVERNANCE

Product release management; Agile product delivery; Portfolio management; OKRs; Product analytics dashboards; Product performance metrics; Customer feedback loops; Product governance; Product compliance; Accessibility; Privacy; Responsible AI considerations; Risk management; Product documentation; Product monitoring; Continuous improvement; Change governance; Regulatory compliance; Product sustainability; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Contemporary tools and practices; Industry case studies.

Text Books/Reference Books:

1. K. Pohl, Requirements Engineering, Springer.
2. S. Robertson and J. Robertson, Mastering the Requirements Process, 3rd ed., Addison-Wesley.
3. M. Cagan, Inspired, 2nd ed., Wiley.
4. T. Gilb, Competitive Engineering, Butterworth-Heinemann.
5. E. Ries, The Lean Startup, Crown Business.

Helankar

Sanjay

P

Distributed Systems and Middleware			
Course Code:	CS647	Course Credits:	3
Course Category:	PEC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3

COURSE OBJECTIVES

1. Explain the principles and terminology of models and communication.
2. Analyse problems and trade-offs concerning coordination and consensus.
3. Apply suitable methods drawn from replication and consistency.
4. Design solutions that integrate transactions and middleware.
5. Evaluate approaches and responsible practice relating to scalable systems.

COURSE OUTCOMES

At the end of the course, the students should be able to:

1. Explain the fundamental concepts and terminology of models and communication.
2. Analyse problems using the methods covered under coordination and consensus.
3. Apply appropriate techniques from replication and consistency to realistic datasets or systems.
4. Design a solution that integrates concepts from transactions and middleware.
5. Evaluate alternative approaches using technical, performance and reliability criteria.
6. Develop and communicate a responsible, reproducible solution drawing on scalable systems.

UNIT I: MODELS AND COMMUNICATION

Architectures; Processes and threads; RPC/RMI; Messaging; Naming; Time and event ordering; Remote invocation semantics; Marshalling; Request-reply protocols; Sockets; Group communication; Publish-subscribe; Message queues; Delivery guarantees; Duplicate suppression; Naming and directory services; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies; Scalability considerations; Ethical and security implications.

UNIT II: COORDINATION AND CONSENSUS

Mutual exclusion; Election; Snapshots; Consensus; Failure detectors; Byzantine faults overview; System assumptions; Safety and progress; Leader election; Replicated state machines; Paxos and Raft concepts; Quorum intersection; Network partitions; Byzantine agreement; Impossibility intuition; Consensus use cases; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies.

UNIT III: REPLICATION AND CONSISTENCY

Data-centric and client-centric consistency; Quorum protocols; Conflict resolution; CAP and PACELC; Primary-backup, multi-primary and quorum replication; Synchronous and asynchronous propagation; Read-your-writes and monotonic reads; Conflict detection and resolution; Anti-entropy; Replication lag; Failover; Worked examples and case analysis; Comparative evaluation of

Handwritten signatures:
 Arulanki
 Jayaraman
 P

Software Maintenance			
Course Code:			
Course Category:	CS649	Course Credits:	3
Course Year (U / P):	PEC	Course (U / P):	P
No. of Lectures + Tutorials (Hrs/Week):	2P	Course Semester (U / P):	3P
	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3

COURSE OBJECTIVES

1. Understand the concepts, processes, and importance of software maintenance.
2. Analyse different software maintenance techniques and reengineering approaches.
3. Apply maintenance practices for software quality improvement and evolution.
4. Evaluate software maintenance strategies, tools, and configuration management.
5. Develop effective maintenance plans for evolving software systems.

COURSE OUTCOMES

At the end of the course, the students should be able to:

1. Explain the concepts and life cycle of software maintenance.
2. Analyse software maintenance processes and maintenance issues.
3. Apply software maintenance and reengineering techniques to software systems.
4. Evaluate software quality, configuration management, and maintenance strategies.
5. Develop maintenance plans for reliable and evolving software applications.
6. Assess software quality and configuration management during maintenance.

UNIT I: INTRODUCTION TO SOFTWARE MAINTENANCE

Introduction to Software Maintenance; Software Evolution; Need and Importance of Maintenance; Types of Maintenance (Corrective, Adaptive, Perfective, Preventive); Software Maintenance Life Cycle; Maintenance Process Models; IEEE/ISO Standards for Software Maintenance; Software Documentation; Legacy Systems; Software Evolution Laws.

UNIT II: SOFTWARE MAINTENANCE TECHNIQUES

Program Comprehension; Reverse Engineering; Forward Engineering; Reengineering; Code Refactoring; Software Restructuring; Data Restructuring; Program Slicing; Software Migration; Software Modernization; Software Reuse; Component-Based Maintenance; Technical Debt.

UNIT III: SOFTWARE QUALITY AND CONFIGURATION MANAGEMENT

Software Quality in Maintenance; Software Testing during Maintenance; Regression Testing; Impact Analysis; Change Management; Software Configuration Management; Version Control Systems; Release Management; Build Management; Continuous Integration; DevOps Practices for Maintenance; Maintenance Metrics.

UNIT IV: MAINTENANCE MANAGEMENT AND AUTOMATION

Maintenance Planning; Maintenance Cost Estimation; Maintenance Team Management; Risk Management; Service Level Agreements (SLAs); Maintenance Tools; Automated Code Analysis; Static and Dynamic Analysis; AI-Assisted Code Review; Bug Tracking Systems; Automated Testing; Continuous Maintenance.

Handwritten signatures and initials:

UNIT V: MODERN SOFTWARE MAINTENANCE PRACTICES

Cloud Application Maintenance; Microservices Maintenance; API Evolution; Containerized Application Maintenance; Security Patch Management; Performance Optimization; Predictive Maintenance using AI; Software Analytics; MLOps for Software Evolution; Green Software Maintenance; Industry Case Studies; Future Trends in Software Maintenance.

Text / Reference Books :

1. Penny Grubb and Armstrong A. Takang, Software Maintenance: Concepts and Practice, World Scientific.
2. Rajlich, V., Software Engineering: The Current Practice, CRC Press.
3. Roger S. Pressman and Bruce Maxim, Software Engineering: A Practitioner's Approach, McGraw-Hill.
4. Ian Sommerville, Software Engineering, Pearson.
5. Martin Fowler, Refactoring: Improving the Design of Existing Code, Addison-Wesley.
6. Michael Feathers, Working Effectively with Legacy Code, Prentice Hall.
7. ISO/IEC/IEEE 14764: Software Engineering – Software Life Cycle Processes – Maintenance.

Helank

Rayson;

[Signature]